

COMPONENTE CURRICULAR:	Jogos Digitais
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

1. INTRODUÇÃO

Este relatório apresenta a análise técnica da aplicação desenvolvida na linguagem Python utilizando a biblioteca Pygame. O objetivo do script é demonstrar a movimentação interativa de uma entidade gráfica (sprite de um peixe-baiacu) em um cenário 2D orientada por cliques do usuário via mouse e calculada por meio de álgebra vetorial.

2. ANÁLISE TÉCNICA E ESTRUTURAL DO CÓDIGO

2.1. Inicialização e Carregamento de Ativos

O sistema inicializa a janela de exibição na resolução de 640×480 pixels. Os ativos visuais são carregados com tratamento otimizado de memória:

- **Background:** Carregado utilizando o método `.convert()` para otimizar a velocidade de renderização no Pygame.
- **Sprite:** Carregado com `.convert_alpha()`, garantindo a preservação do canal de transparência (Alpha) da imagem sobre o fundo.

2.2. Matemática de Vetores (**Vector2**) e Vetor Direção

Para que o peixe navegue até a posição desejada, o script faz uso da classe **Vector2** da biblioteca `gameobjects`:

1. **Captura do Evento:** No evento `MOUSEBUTTONDOWN`, a posição do clique é ajustada subtraindo metade das dimensões da imagem do peixe (`get_width()/2.0` e `get_height()/2.0`), garantindo que o centro da imagem vá até o destino.
2. **Cálculo do Vetor Direção:** O vetor `heading` é instanciado através de `Vector2.from_points(position, destination)`, traçando a linha de deslocamento entre a posição atual e a final.
3. **Normalização:** A chamada ao método `heading.normalize()` transforma o vetor em um vetor unitário (com magnitude igual a 1). Isso isola unicamente a direção do movimento, permitindo multiplicar a trajetória pela velocidade desejada sem alterar o ritmo.

2.3. Controle do Tempo e Velocidade Independente (Delta Time)

Para assegurar que o movimento ocorra a uma velocidade constante de 250 pixels por segundo (`speed = 250`), independentemente da capacidade computacional ou da variação da taxa de quadros (FPS):

- O método `clock.tick()` retorna o tempo transcorrido em milissegundos desde o último frame.
- O tempo é convertido para segundos (`time_passed_seconds = time_passed / 1000.0`).
- A distância percorrida no frame é dada pelo cálculo do *Delta Time*:
 $distância = tempo \times velocidade$.
- A nova posição do sprite é atualizada dinamicamente pela equação vetorial
`position += heading * distance_moved`.

2.4. Renderização e Atualização da Tela

A cada ciclo da execução do *loop* principal:

1. O fundo é redesenhado cobrindo o frame anterior (`screen.blit(background, (0,0))`).
2. O peixe é desenhado na nova coordenada vetorial (`screen.blit(sprite, (position.x, position.y))`).
3. A chamada a `pygame.display.update()` atualiza o frame na tela para o jogador.

3. CONCLUSÃO

A atividade permitiu observar na prática como o controle de animação e movimentação 2D em jogos digitais depende de conceitos fundamentais da matemática vetorial e da física computacional. O uso da normalização de vetores combinada com a medição do tempo decorrido (*Delta Time*) mostra-se a abordagem ideal para construir movimentações fluidas, previsíveis e independentes do desempenho do hardware.